SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

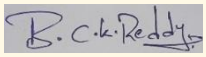
Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	B. Charan Kumar Reddy	**Reg. No.:**	192472195
Date:	26/10/2026	**Duration:**	60 minutes

Code of Conduct

I B. Charan Kumar Reddy (192472195) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: 

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.

- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
    - Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Answers

Problem Statement

The Autonomous Coding Agent Security and Software Supply-Chain Protection Platform is designed to monitor AI coding agents, source-code repositories, third-party dependencies, and Continuous Integration/Continuous Deployment (CI/CD) workflows. As AI coding agents become capable of modifying code, installing dependencies, and executing development tools, they introduce security risks such as unauthorized tool execution, privilege escalation, secret exposure, malicious dependency updates, and abnormal repository modifications.

The proposed platform identifies suspicious activities, calculates risk scores, generates security alerts, maintains a Software Bill of Materials (SBOM), and records security events in audit logs. It also enforces security policies, supports human approval for sensitive actions, and enables the isolation or rollback of suspicious changes. The architecture focuses on security, scalability, reliability, maintainability, and integration with existing software development environments.

1. ANALYZE SYSTEM REQUIREMENTS

a) System Objectives

The primary objective of the proposed system is to provide a centralized security platform that protects AI-assisted software development and the software supply chain.

The major objectives are:

1. To continuously monitor AI coding agent activities and identify unauthorized or suspicious operations.
2. To track source-code repository modifications and detect abnormal changes.
3. To scan third-party dependencies for known vulnerabilities, suspicious versions, and unauthorized changes.
4. To generate and maintain a Software Bill of Materials (SBOM) containing information about software components and dependencies.
5. To monitor CI/CD pipelines and identify suspicious build activities.
6. To detect exposed secrets, privilege escalation, and unauthorized tool execution.
7. To calculate risk scores and generate alerts based on detected security events.
8. To enforce security policies and require human approval for sensitive operations.
9. To isolate suspicious changes and support rollback to a previously verified state.
10. To maintain centralized audit logs and reports for security monitoring, investigation, and compliance.

b) Functional Requirements

Functional requirements describe the operations and services that the system must provide.

- **FR1 – AI-Agent Monitoring:** The system shall monitor AI coding agent actions, tool usage, permissions, and execution requests.
- **FR2 – Repository Monitoring:** The system shall track repository changes, including code modifications, configuration changes, and dependency-file updates.
- **FR3 – Dependency Scanning:** The system shall scan project dependencies and identify known vulnerabilities, suspicious versions, and policy violations.
- **FR4 – SBOM Management:** The system shall generate and update an SBOM containing software components, versions, and dependency relationships.
- **FR5 – Secret Detection:** The system shall identify potential exposure of API keys, passwords, tokens, and other sensitive information.
- **FR6 – CI/CD Monitoring:** The system shall monitor build and deployment workflows for suspicious commands, unauthorized changes, and policy violations.
- **FR7 – Risk Assessment:** The system shall calculate risk scores using security findings, severity, permissions, and activity context.
- **FR8 – Alert Generation:** The system shall notify authorized users when suspicious or high-risk activities are detected.
- **FR9 – Human Approval:** The system shall allow authorized users to approve or reject sensitive AI-agent actions.
- **FR10 – Threat Isolation:** The system shall support isolating suspicious changes, agent sessions, or affected workflows when required.
- **FR11 – Rollback Management:** The system shall support rollback of unsafe changes to a previously verified state when recovery is possible.
- **FR12 – Audit and Policy Management:** The system shall maintain audit logs, generate reports, and allow authorized administrators to manage security policies.

c) Non-Functional Requirements

Non-functional requirements define the quality, performance, and operational constraints of the platform.

- **Scalability:** The system should support increasing numbers of users, repositories, AI agents, dependencies, and security events.
- **Security:** Authentication, role-based access control, encryption, least-privilege permissions, and secure communication must protect the platform.
- **Performance:** The system should process monitoring events and produce risk assessments with minimal delay.
- **Reliability:** Security events should be processed consistently, and failures should not silently result in lost critical events.
- **Availability:** The monitoring dashboard and critical security services should remain available during normal operation and recover quickly from failures.
- **Maintainability:** The system should use modular components so that individual services can be updated without redesigning the entire platform.
- **Usability:** The dashboard should present security alerts, risk levels, audit records, and approval requests clearly.

- **Interoperability:** The platform should integrate with supported source repositories, dependency registries, AI coding agents, and CI/CD systems through APIs, webhooks, or compatible connectors.
- **Auditability:** Security-sensitive operations should be recorded with timestamps, identities, actions, and outcomes.
- **Recoverability:** The platform should support restoring its own data and recovering affected development workflows using verified backups and recovery procedures.

d) Quality Attributes Influencing Architecture

The most important quality attributes are security, scalability, reliability, availability, performance, and maintainability.

Security is essential because the platform handles source-code activity, sensitive credentials, dependency information, and privileged agent operations. Scalability is necessary because monitoring workloads can increase as more repositories and agents are connected. Reliability and availability ensure that critical security events are processed and communicated. Performance reduces the delay between detecting a threat and responding to it. Maintainability allows new integrations, detection rules, and security capabilities to be added without major architectural changes.

2. SELECT A SUITABLE SOFTWARE ARCHITECTURE

a) Selected Architecture: Hybrid Layered and Event-Driven Architecture

The proposed system uses a **Hybrid Layered and Event-Driven Architecture**. The layered structure separates the user interface, API and access control, application services, security processing, and data storage. The event-driven mechanism allows monitoring components to publish security events that can be processed asynchronously by detection, risk assessment, alerting, and audit services.

This architecture is suitable because the platform must integrate with several external systems, process different types of security events, and respond to suspicious activities without tightly coupling every component.

b) Architectural Layers

The proposed architecture consists of the following layers.

1. Presentation Layer

This layer contains the web dashboard used by developers, security administrators, and DevOps engineers. It displays alerts, risk scores, repository activity, SBOM information, approval requests, audit logs, and security reports.

2. API and Access-Control Layer

This layer provides the API gateway or backend API entry point. It handles request routing, authentication, authorization, input validation, rate limiting, and communication between the dashboard and backend services.

3. Integration and Event Collection Layer

This layer connects the platform to AI coding agents, source-code repositories, dependency registries, and CI/CD pipelines. It collects activity through supported APIs, webhooks, and connectors, then converts incoming information into standardized security events.

4. Security Analysis Layer

This layer contains the monitoring and analysis services. It detects suspicious agent actions, exposed secrets, abnormal repository changes, vulnerable dependencies, and unusual CI/CD activities. It also evaluates security policies and calculates risk scores.

5. Response and Governance Layer

This layer manages security alerts, human approval requests, policy enforcement, suspicious-change isolation, and rollback operations. Sensitive or potentially destructive actions require suitable authorization and verification.

6. Data Management Layer

This layer stores user and policy information, repository metadata, security findings, SBOM records, audit logs, and alert history. Relational storage can be used for structured records, while object storage can hold SBOM documents, reports, and other artifacts.

7. Event Processing Mechanism

An event broker or message queue connects monitoring services with downstream analysis and audit services. It enables asynchronous processing, buffering during temporary workload spikes, and independent scaling of consumers.

c) Justification for Selecting the Architecture

The hybrid architecture is appropriate for the following reasons:

1. **Separation of responsibilities:** Each layer has a clearly defined role, making the platform easier to understand, develop, test, and maintain.
2. **Loose coupling:** Event producers and consumers communicate through standardized events, reducing direct dependencies between monitoring and analysis components.
3. **Scalability:** High-demand components, such as dependency scanning and risk analysis, can be scaled independently when implemented as separate services.
4. **Security:** Authentication, authorization, policy enforcement, and sensitive-action approval can be handled through dedicated components.
5. **Integration:** API-based connectors and event collection support integration with multiple development tools and external systems.
6. **Reliability:** Queues, retries, health checks, and durable event storage can help prevent event loss and support recovery.
7. **Extensibility:** New monitoring rules, integrations, and response mechanisms can be added without restructuring the complete platform.

A purely layered architecture could make asynchronous event processing more difficult to scale independently. A fully distributed microservices architecture could introduce unnecessary deployment and operational complexity. The hybrid approach balances modularity, event processing, and manageable implementation complexity.

3. DRAW THE SOFTWARE ARCHITECTURE DIAGRAM

a) Overall Architecture

The architecture diagram represents the major components, their communication paths, the system's data stores, and the external systems that interact with the platform.

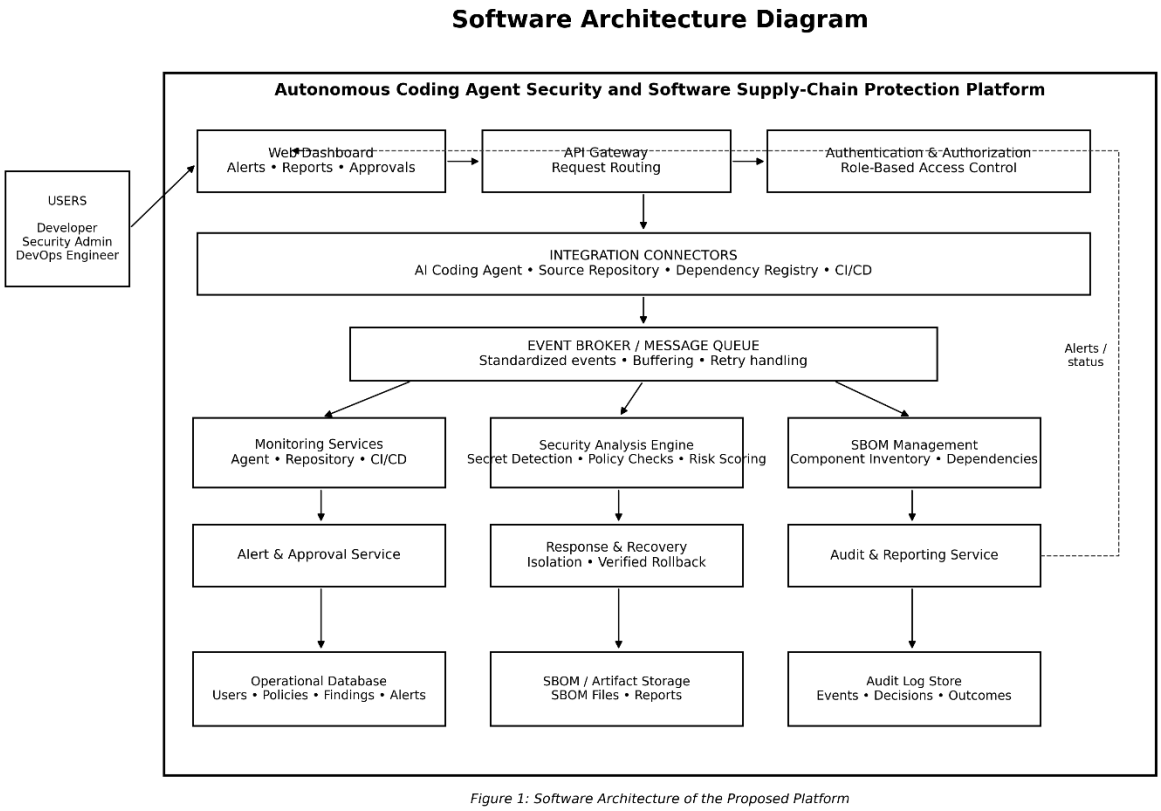


Figure 1: Software Architecture of the Proposed Platform

Figure 1: Software Architecture of the Autonomous Coding Agent Security and Software Supply-Chain Protection Platform.

The diagram should contain the following components:

1. **Users:** Developer, Security Administrator, and DevOps Engineer.
2. **Web Dashboard:** Displays alerts, reports, SBOM information, repository activity, and approval requests.
3. **API Gateway and Authentication Service:** Validates requests, checks permissions, and routes requests to backend services.
4. **Integration Connectors:** Connect the platform to AI coding agents, source-code repositories, dependency registries, and CI/CD pipelines.
5. **Event Broker / Message Queue:** Transfers monitoring events to the appropriate processing services.

6. **Monitoring Services:** Collect and analyze agent activity, repository changes, dependency information, and CI/CD events.
7. **Security Analysis Engine:** Performs secret detection, policy evaluation, vulnerability analysis, and risk scoring.
8. **SBOM Management Service:** Generates, stores, and updates software component inventories.
9. **Alert and Approval Service:** Sends notifications and manages human approval or rejection of sensitive actions.
10. **Response and Recovery Service:** Coordinates isolation and rollback operations after authorization and verification.
11. **Audit and Reporting Service:** Records security events, decisions, response actions, and report information.
12. **Databases and Storage:** Store user records, security policies, repository metadata, alerts, findings, audit logs, and SBOM artifacts.

The diagram should show data flowing from external systems through the integration connectors to the event-processing and security-analysis components. The results should be stored in the appropriate data stores and displayed through the dashboard. High-risk events should trigger alerts and, when necessary, approval or response workflows.

4. EXPLAIN COMPONENTS AND THEIR INTERACTIONS

a) Component Responsibilities

1. Web Dashboard

The dashboard provides the main interface for users. Developers can view repository security findings and alerts, security administrators can manage policies and approvals, and DevOps engineers can monitor CI/CD security events and recovery status.

2. API Gateway and Authentication Service

The API gateway acts as the entry point for dashboard requests and supported integrations. It validates incoming requests, applies authentication and authorization checks, and forwards requests to the appropriate services. Role-based access control restricts sensitive operations to authorized users.

3. Integration Connectors

Connectors retrieve information from supported AI coding agents, source repositories, dependency registries, and CI/CD pipelines. They collect events such as agent tool execution, repository commits, dependency changes, and build activity. Incoming data is validated and converted into a standardized event format.

4. Event Broker / Message Queue

The event broker distributes collected events to the appropriate monitoring and analysis services. It separates event collection from event processing, allowing services to process events asynchronously.

Queue buffering, retry mechanisms, and durable messaging can improve resilience during temporary failures.

5. AI-Agent Monitoring Service

This service tracks agent actions, tool execution requests, permission usage, and other available activity records. It compares observed actions with defined policies and identifies suspicious or unauthorized behavior.

6. Repository and Dependency Monitoring Services

The repository monitoring service detects unusual code modifications, configuration changes, and dependency-file updates. The dependency scanning service examines dependency names, versions, vulnerability information, and policy compliance using available registry data and security feeds.

7. Security Analysis and Risk Scoring Engine

This component combines monitoring findings and security rules to identify suspicious activity. It evaluates factors such as vulnerability severity, action sensitivity, permission level, and event context. It produces a risk score and assigns a suitable severity level to support prioritization.

8. SBOM Management Service

The SBOM service creates and updates an inventory of software components and their dependency relationships. It records available component names, versions, and relevant metadata, helping security teams understand which dependencies are present in a project.

9. Alert and Approval Service

This service generates notifications when security findings require attention. For sensitive operations, it creates approval requests for authorized users and records whether the action was approved or rejected.

10. Response and Recovery Service

This component coordinates isolation and rollback procedures. It can suspend or restrict a suspicious workflow, quarantine an affected artifact, or initiate rollback to a previously verified state. Before execution, it checks permissions and applicable policies. The system records the response and its outcome.

11. Audit and Reporting Service

This service maintains records of detected events, risk assessments, approval decisions, policy changes, and response actions. It generates reports that support incident investigation, security reviews, and accountability.

12. Data Management Components

The database stores structured records such as users, policies, repository metadata, findings, alerts, and approval decisions. Object storage can be used for SBOM files, reports, and other artifacts. Audit records should have suitable access controls and integrity protections.

b) Communication and Data Exchange

The system uses REST APIs, webhooks, event messages, and database operations to exchange information.

- The dashboard communicates with backend services through authenticated API requests.
- Integration connectors receive events from external systems through supported APIs and webhooks.
- Connectors publish standardized events to the event broker.
- Monitoring services consume events and generate security findings.
- The security analysis engine evaluates findings and calculates risk scores.
- The alert service notifies authorized users when action is required.
- The approval service returns decisions to the relevant workflow.
- The response service coordinates approved isolation or rollback operations.
- The SBOM service updates component inventory records when relevant dependency information changes.
- The audit service records important events, decisions, and outcomes in persistent storage.

Communication between components should use authenticated channels, validated data formats, and appropriate access permissions. Events should include identifiers, timestamps, source information, event types, and correlation identifiers where applicable.

c) Overall System Workflow

The complete workflow is as follows:

1. A developer works with an AI coding agent, source-code repository, or CI/CD pipeline.
2. The integration connectors collect supported activity events and relevant dependency information.
3. The event broker distributes the collected events to the appropriate monitoring services.
4. The monitoring services inspect agent actions, repository modifications, dependency changes, and build activities.
5. The security analysis engine checks the findings against security policies and calculates risk scores.
6. The SBOM service generates or updates the inventory when relevant software components or dependencies change.
7. If a security issue is detected, the alert service creates a notification and records the finding.
8. If a sensitive action requires approval, the system sends an approval request to an authorized user.
9. After authorization and policy checks, the response service performs the permitted isolation or rollback operation when required.
10. The audit service records the detection, risk assessment, approval decision, and response outcome.
11. Users access the dashboard to review alerts, audit logs, SBOM records, and security reports.

This workflow provides continuous monitoring and a structured response process while maintaining traceability of important security operations.

5. DISCUSS QUALITY ATTRIBUTES

a) Scalability

The event-driven design allows monitoring and analysis components to process events independently. If dependency scanning or risk analysis becomes a bottleneck, additional service instances can be deployed to handle increased demand. The event broker can buffer incoming events during temporary workload spikes.

Database indexing, efficient queries, pagination, and suitable storage strategies can improve performance as the volume of repositories, alerts, and audit records increases. Scaling decisions should be based on workload measurements and performance testing.

b) Security

Security is a primary requirement because the platform monitors privileged development operations and sensitive software information.

The architecture supports security through:

- Authentication for users and integrated services.
- Role-based access control and least-privilege permissions.
- Encryption of data in transit and appropriate protection of stored data.
- Secure storage and restricted access to credentials and secrets.
- Input validation and API access controls.
- Security-policy enforcement and human approval for sensitive actions.
- Audit logging of administrative activities and security responses.
- Isolation of suspicious workflows and controlled rollback procedures.

The platform should not store exposed secrets unnecessarily in alerts or logs. Security findings should contain only the information needed for investigation, with sensitive values redacted where possible.

c) Performance

The event-driven mechanism allows monitoring events to be processed asynchronously rather than requiring every operation to wait for all analysis tasks to finish. Independent processing services can handle different categories of security events.

Database indexing, caching of appropriate reference data, efficient scanning, and controlled concurrency can reduce processing time. Time-sensitive events should be prioritized according to severity. Performance testing should measure event-processing latency, alert-generation time, API response time, and throughput.

d) Reliability

Reliability is improved through modular components, event persistence, retry mechanisms, error handling, and health monitoring. If a downstream service temporarily fails, the event broker can retain supported messages until processing resumes, depending on its configuration.

Services should detect duplicate events and use suitable idempotent processing where repeated delivery is possible. Failed events should be recorded and routed for investigation rather than silently discarded. Database backups and tested recovery procedures should protect important records.

e) Maintainability

The layered and modular design separates presentation, integration, monitoring, analysis, response, and storage responsibilities. Developers can modify a security rule or connector without redesigning the entire system.

Standard APIs, documented event formats, automated tests, centralized configuration, and version control improve maintainability. The platform can introduce additional integrations, detection techniques, and reporting features through well-defined component interfaces.

f) Availability

The system can improve availability by monitoring component health, restarting failed services, using redundant instances for critical components, and backing up important data. The event broker can help prevent temporary downstream outages from immediately interrupting event collection.

Critical functions such as event intake, alerting, and audit recording should receive suitable operational priorities. Availability also depends on infrastructure configuration, database resilience, deployment strategy, and recovery testing.

6. JUSTIFY ARCHITECTURAL DECISIONS

a) Rationale Behind Architectural Choices

The hybrid layered and event-driven architecture was selected because the platform must monitor several independent external systems, analyze high volumes of security events, and coordinate security responses.

The layered design provides clear separation of responsibilities and simplifies development and testing. The event-driven mechanism enables asynchronous processing and reduces direct coupling between event producers and consumers. Modular services allow resource-intensive operations, such as dependency scanning and risk assessment, to scale independently when needed.

A centralized API entry point provides a consistent location for authentication, authorization, and request validation. Separate data management components allow structured records, audit information, and SBOM artifacts to be stored according to their requirements.

These choices support the platform's major objectives of security monitoring, threat detection, traceability, controlled response, and future integration.

b) Architectural Trade-Offs

Advantages

1. Modular components improve maintainability and support independent development.

2. Event-driven processing supports asynchronous workloads and reduces direct dependencies.
3. Independent service scaling can improve resource utilization.
4. Centralized access control and policy enforcement improve governance.
5. Separate audit and SBOM services improve traceability and software inventory management.
6. Standard APIs and connectors support integration with different development environments.

Limitations

1. Event-driven systems introduce additional complexity in message handling, retries, ordering, and duplicate-event processing.
2. Multiple services require more monitoring, deployment, configuration, and operational management than a simple monolithic application.
3. Distributed communication may introduce network latency and partial failures.
4. Reliable integration depends on the capabilities and availability of external APIs and tools.
5. Automatic isolation or rollback can disrupt development workflows if detection results are inaccurate or recovery procedures are not verified.
6. Security analysis and dependency scanning can consume significant processing resources as the workload grows.

Mitigation Strategies

These trade-offs can be managed through standardized event schemas, idempotent processing, monitoring, retry limits, dead-letter queues, automated testing, and documented recovery procedures. High-impact actions should use policy checks and suitable approval controls. The platform should validate rollback targets before attempting recovery.

c) Support for Future Enhancements

The architecture can be extended to support new security requirements without replacing the complete platform.

1. **Advanced Threat Detection:** Additional rule-based or machine-learning detection modules can be introduced through the security analysis layer.
2. **New AI-Agent Integrations:** New connectors can be added to monitor additional AI coding tools and their supported events.
3. **Expanded Dependency Intelligence:** Additional vulnerability feeds, package registries, and software composition analysis tools can be integrated.
4. **Enhanced SBOM Support:** The platform can support additional SBOM formats and improved component relationship tracking.
5. **Advanced Reporting:** New dashboards, analytics, and compliance reports can be added using the audit and reporting services.
6. **Policy Automation:** Security administrators can introduce configurable rules for agent permissions, dependency approvals, and CI/CD execution.
7. **Cloud Deployment:** Components can be deployed on cloud infrastructure with appropriate scaling, monitoring, and security controls.
8. **Improved Recovery:** Additional quarantine mechanisms, verified recovery points, and controlled rollback strategies can be incorporated.

d) System Integration and Long-Term Maintainability

The platform uses defined APIs and integration connectors to communicate with external tools. This approach reduces dependence on any single repository provider, dependency registry, or CI/CD platform, provided compatible interfaces are available.

Long-term maintainability can be supported through modular implementation, automated testing, version-controlled configuration, documented interfaces, secure dependency management, and regular security reviews. Each component can be upgraded independently when interfaces and data contracts remain compatible.

The architecture also supports gradual implementation. A first version can provide basic monitoring, dependency scanning, SBOM generation, alerts, and audit logging. Later versions can introduce more advanced risk analysis, broader integrations, and automated response features after appropriate testing.

Conclusion

The Autonomous Coding Agent Security and Software Supply-Chain Protection Platform addresses the growing security risks associated with AI-assisted software development. The proposed hybrid layered and event-driven architecture separates system responsibilities, supports asynchronous security-event processing, and enables integration with AI coding agents, repositories, dependency registries, and CI/CD pipelines.

The architecture provides dedicated components for monitoring, security analysis, SBOM management, alerting, approval, response, audit logging, and data storage. Its design addresses the quality attributes of scalability, security, performance, reliability, maintainability, and availability while acknowledging the complexity of distributed processing and automated recovery.

Overall, the proposed architecture provides a modular and extensible foundation for protecting software development workflows and improving visibility into software supply-chain risks.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope;	Justification is weak or absent; report lacks organization and technical clarity.

	presents a well-organized, professional report.		report needs improvement.	

Criteria	Mark
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25